

AI-Driven API Testing Platform

Database Schema Design Report (BCNF)

Version: 1.0

Prepared For: ATP AI Platform

Architecture Type: Relational + JSONB Hybrid Design

Database: PostgreSQL + pgvector

Normalization Standard: BCNF (Boyce-Codd Normal Form)

1. Introduction

This document defines the database schema design for the AI-Driven API Testing Platform.

The platform automatically:

- imports Swagger/OpenAPI specifications
- detects roles and authentication flows
- executes API testing workflows
- manages sessions and JWT tokens
- supports OTP authentication workflows
- stores API testing results
- generates AI-powered reports

The schema is designed using:

- relational database principles
- BCNF normalization
- scalable enterprise architecture
- JSONB support for dynamic API schemas

The design focuses on:

- scalability
 - maintainability
 - query efficiency
 - clean relationships
 - extensibility for AI workflows
-

2. Why PostgreSQL?

PostgreSQL is selected because:

- ACID compliant
- enterprise-grade reliability
- excellent relational support
- native JSONB support
- pgvector support for AI embeddings
- indexing support
- high scalability
- strong concurrency handling

The platform requires both:

- structured relational data
- flexible API schema storage

PostgreSQL is ideal for this hybrid requirement.

3. Why BCNF?

BCNF (Boyce-Codd Normal Form) is used to:

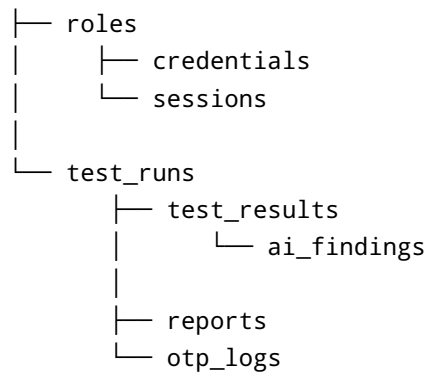
- eliminate redundancy
- avoid update anomalies
- avoid insertion anomalies
- avoid deletion anomalies
- maintain consistency
- improve maintainability

The schema ensures:

- every determinant is a candidate key
 - non-key attributes fully depend on primary keys
 - no transitive dependencies
-

4. High-Level Database Architecture

```
users
├── projects
│   └── endpoints
```



5. Core Entities

The platform contains the following core entities:

Entity	Purpose
users	Platform users
projects	API testing projects
endpoints	Imported Swagger APIs
roles	Role definitions
credentials	Login credentials per role
sessions	JWT/session management
test_runs	Individual testing executions
test_results	API execution results
reports	Generated reports
ai_findings	AI-generated analysis
otp_logs	OTP workflow tracking

6. Schema Design

6.1 users

Purpose: Stores ATP AI platform users.

Attributes

Column	Type	Constraints
id	UUID	PK
name	VARCHAR(100)	NOT NULL
email	VARCHAR(255)	UNIQUE, NOT NULL
password_hash	TEXT	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Functional Dependency

```
id → name, email, password_hash
email → id, name, password_hash
```

BCNF Validation

All determinants are candidate keys.

Table satisfies BCNF.

6.2 projects

Purpose: Stores API testing projects.

Attributes

Column	Type	Constraints
id	UUID	PK
user_id	UUID	FK → users.id
name	VARCHAR(255)	NOT NULL
swagger_url	TEXT	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()

Functional Dependency

`id → user_id, name, swagger_url`

Relationship

One User → Many Projects

BCNF Validation

No partial or transitive dependency exists.

Table satisfies BCNF.

6.3 endpoints

Purpose: Stores imported Swagger/OpenAPI endpoints.

This is one of the most important tables in the platform because the AI testing engine depends heavily on API schema structures.

Dynamic API structures are stored using JSONB.

Attributes

Column	Type	Constraints
id	UUID	PK
project_id	UUID	FK → projects.id
method	VARCHAR(10)	NOT NULL
path	TEXT	NOT NULL
summary	TEXT	NULL
request_schema	JSONB	NULL
response_schema	JSONB	NULL
headers_schema	JSONB	NULL
query_schema	JSONB	NULL

Column	Type	Constraints
path_params_schema	JSONB	NULL
auth_required	BOOLEAN	DEFAULT FALSE
tags	JSONB	NULL
created_at	TIMESTAMP	DEFAULT NOW()

Why Schema Fields Are Required

The platform is not only storing endpoint URLs.

It stores complete API behavior models used by:

- AI payload generation
- schema validation
- negative testing
- authentication testing
- chained API execution
- smart report generation

Example request_schema

```
{
  "type": "object",
  "properties": {
    "name": {
      "type": "string"
    },
    "number": {
      "type": "string"
    }
  }
}
```

Functional Dependency

```
id → all attributes
(project_id, method, path) → all attributes
```

BCNF Validation

Composite endpoint uniqueness ensures no duplicate APIs.

Table satisfies BCNF.

6.4 roles

Purpose: Stores roles detected or configured for a project.

Examples:

- Admin
- Client
- Worker
- Platform Admin

Attributes

Column	Type	Constraints
id	UUID	PK
project_id	UUID	FK → projects.id
name	VARCHAR(100)	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()

Functional Dependency

```
id → project_id, name
(project_id, name) → id
```

BCNF Validation

No transitive dependency exists.

Table satisfies BCNF.

6.5 credentials

Purpose: Stores target API authentication credentials per role.

This table does not store ATP AI platform users. It stores credentials required to authenticate against the imported target APIs.

Examples:

- Admin login
- Client login
- Worker login

Attributes

Column	Type	Constraints
id	UUID	PK
role_id	UUID	FK → roles.id
username	VARCHAR(255)	NOT NULL
password_encrypted	TEXT	NOT NULL
login_endpoint	TEXT	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()

Why password_encrypted Is Required

Target API credentials may contain highly sensitive access data. Passwords must never be stored in plain text.

The system should encrypt credentials before database storage.

Why login_endpoint Is Required

Different APIs use different authentication endpoints.

Examples:

- /auth/login
- /api/login
- /oauth/token
- /signin

The testing engine must know which endpoint to call to obtain authentication tokens.

Functional Dependency

id → role_id, username, password_encrypted, login_endpoint

BCNF Validation

Table satisfies BCNF.

6.6 sessions

Purpose: Stores JWT/session tokens.

Supports:

- multiple roles
- multiple concurrent sessions
- token expiration
- refresh flows

Attributes

Column	Type	Constraints
id	UUID	PK
role_id	UUID	FK → roles.id
access_token	TEXT	NOT NULL
refresh_token	TEXT	NULL
expires_at	TIMESTAMP	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()

Functional Dependency

id → role_id, access_token, expires_at

BCNF Validation

Table satisfies BCNF.

6.7 test_runs

Purpose: Represents one complete API testing execution.

Attributes

Column	Type	Constraints
id	UUID	PK
project_id	UUID	FK → projects.id
status	VARCHAR(50)	NOT NULL
total_apis	INTEGER	DEFAULT 0
passed_count	INTEGER	DEFAULT 0
failed_count	INTEGER	DEFAULT 0
started_at	TIMESTAMP	DEFAULT NOW()
completed_at	TIMESTAMP	NULL

Example Status

```
running
completed
failed
```

Functional Dependency

```
id → all attributes
```

BCNF Validation

Table satisfies BCNF.

6.8 test_results

Purpose: Stores the result of every API execution.

This table will grow significantly over time.

Attributes

Column	Type	Constraints
id	UUID	PK
run_id	UUID	FK → test_runs.id
endpoint_id	UUID	FK → endpoints.id
role_id	UUID	FK → roles.id
status_code	INTEGER	NOT NULL
response_body	JSONB	NULL
response_headers	JSONB	NULL
response_time_ms	INTEGER	NULL
success	BOOLEAN	DEFAULT FALSE
error_message	TEXT	NULL
created_at	TIMESTAMP	DEFAULT NOW()

Functional Dependency

id → all attributes

BCNF Validation

No redundancy exists.

Table satisfies BCNF.

6.9 reports

Purpose: Stores generated PDF/HTML report references.

Generated reports are typically stored in:

- local storage
- object storage
- cloud storage

The database stores only the report location reference.

Attributes

Column	Type	Constraints
id	UUID	PK
run_id	UUID	FK → test_runs.id
report_url	TEXT	NOT NULL
summary	TEXT	NULL
created_at	TIMESTAMP	DEFAULT NOW()

Why report_url Is Required

Storing large PDF or HTML binaries directly inside the database is inefficient.

The recommended architecture is:

- generate report file
- store file externally
- save report location/path in database

Functional Dependency

id → run_id, report_url, summary

BCNF Validation

Table satisfies BCNF.

6.10 ai_findings

Purpose: Stores AI-generated findings and recommendations.

Attributes

Column	Type	Constraints
id	UUID	PK
result_id	UUID	FK → test_results.id

Column	Type	Constraints
issue	TEXT	NOT NULL
suggestion	TEXT	NOT NULL
severity	VARCHAR(50)	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()

Why severity Is Required

AI findings can have different risk levels.

Examples:

Severity	Example
Low	Slow response time
Medium	Missing validation
High	Authentication issue
Critical	Unauthorized admin access

Severity allows:

- prioritization of issues
- frontend risk indicators
- AI risk scoring
- security alert generation

Example

Issue:
403 Forbidden

Suggestion:
Verify RBAC middleware and JWT attachment.

Functional Dependency

id → result_id, issue, suggestion, severity

BCNF Validation

Table satisfies BCNF.

6.11 otp_logs

Purpose: Tracks OTP authentication workflows.

This table supports one of the platform's advanced features.

Attributes

Column	Type	Constraints
id	UUID	PK
run_id	UUID	FK → test_runs.id
role_id	UUID	FK → roles.id
otp_value	VARCHAR(10)	NULL
source	VARCHAR(50)	NOT NULL
status	VARCHAR(50)	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()

Example Sources

```
terminal
email
sms
api_response
```

Functional Dependency

```
id → run_id, role_id, otp_value, source, status
```

BCNF Validation

Table satisfies BCNF.

7. Relationship Model

One-to-Many Relationships

Parent	Child
users	projects
projects	endpoints
projects	roles
roles	credentials
roles	sessions
projects	test_runs
test_runs	test_results
test_results	ai_findings
test_runs	reports
test_runs	otp_logs

8. Why JSONB Is Used

JSONB is used because Swagger/OpenAPI schemas are dynamic.

Different APIs contain different payload structures.

Examples:

```
{
  "name": "Ali",
  "number": "989898"
}
```

```
{
  "productId": 1,
  "price": 500
}
```

```
{
  "otp": "565656"
}
```

Using relational columns for all payloads is impossible.

JSONB provides:

- flexibility
- indexing support
- high-performance querying
- schema evolution support

9. Why UUID Primary Keys?

UUIDs are preferred because:

- globally unique
- microservice-friendly
- secure for public APIs
- safer for distributed systems
- avoids predictable IDs

10. Indexing Strategy

Recommended indexes:

Table	Index
users	email
projects	user_id
endpoints	project_id
endpoints	method + path
roles	project_id
test_runs	project_id
test_results	run_id
ai_findings	result_id

Table	Index
sessions	role_id

11. Security Considerations

JWT Security

- store tokens securely
- use short-lived sessions
- avoid localStorage for frontend tokens

Credential Encryption

- encrypt passwords at rest
- use bcrypt/argon2

API Data Validation

- validate imported Swagger schemas
 - sanitize inputs
 - validate JSON structures
-

12. Scalability Considerations

The schema is designed for:

- parallel API testing
- distributed workers
- large-scale test execution
- AI analysis expansion
- report generation
- vector embedding support

Future scalability:

- read replicas
 - partitioning
 - sharding
 - caching
 - queue scaling
-

13. Future Enhancements

Future tables may include:

Table	Purpose
audit_logs	user activity tracking
api_dependencies	endpoint chaining
environments	staging/prod configs
vector_embeddings	AI semantic memory
notifications	email/slack alerts
schedules	automated testing jobs

14. Conclusion

The ATP AI database schema is designed using BCNF principles to ensure:

- minimal redundancy
- high scalability
- strong maintainability
- enterprise-grade architecture
- AI-readiness
- secure session handling
- dynamic API schema support

The combination of:

- PostgreSQL
- JSONB
- UUIDs
- pgvector
- normalized relational design

creates a strong foundation for building a scalable AI-driven API testing platform.